Reviewer Pack — Tutte Polynomial Evaluation Bounded by Monotone k-CLIQUE Cir…

Entry 954a5c8e8ffd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 01:05:26 UTC

Tutte Polynomial Evaluation Bounded by Monotone k-CLIQUE Circuit Size

Entry ID: `954a5c8e8ffd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-11 01:05:26 UTC

1. Conjecture

Field A (mathematical branch): Tutte Polynomial Theory **Field B** (complexity object): Monotone Circuit Size for k-CLIQUE

Statement:

> For any k-CLIQUE instance on n vertices, the Tutte polynomial evaluated at (2, 1) satisfies T(2,1) ≥ Ω(n^{k/2}) and T(1,2) ≥ Ω(n^{k/2}), where T(x,y) is the Tutte polynomial of the graphic matroid induced by the complete graph K_n.

Rationale (proposer's reasoning):

> The Tutte polynomial encodes matroid structure, and its evaluation at (2,1) and (1,2) captures connectivity properties. For k-CLIQUE, these values may correlate with the exponential growth of monotone circuit size due to the combinatorial explosion in clique enumeration.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1e3bc00a50d93df2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from collections import defaultdict

def generate_random_graph(n):
    graph = defaultdict(set)
    for i in range(n):
        for j in range(i + 1, n):
            if random.choice([True, False]):
                graph[i].add(j)
                graph[j].add(i)
    return graph

def tutte_polynomial(graph, memo=None):
    if memo is None:
        memo = {}
    if (graph,) in memo:
        return memo[(graph,)]

    n = len(graph)
    if n == 0:
        return 1
    if n == 1:
        return 2

    v = next(iter(graph))
    edges = [(v, u) for u in graph[v]]
    T = 0
    for e in edges:
        u, w = e
        G_uw = {u: set(), w: set()}
        for x in graph[u]:
            if x != w:
                G_uw[u].add(x)
        for y in graph[w]:
            if y != u:
                G_uw[w].add(y)
        for x in graph[u]:
            for y in graph[w]:
                if x != y and (x, y) not in edges:
```

```
                    G_uw[x].add(y)

        T += tutte_polynomial(G_uw, memo)

    memo[(graph,)] = T
    return T

def get_tutte_values(graph, k):
    T21 = tutte_polynomial(graph)
    T12 = tutte_polynomial({u: {v for v in graph[u] if v > u} for u in range(k)})
    return T21, T12

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        graph = generate_random_graph(n)
        T21, T12 = get_tutte_values(graph, n // 2 + 1)

        if T21 < n ** (n // 4) or T12 < n ** (n // 4):
            return {
                "metric_name": "T(2,1), T(1,2)",
                "metric_value": None,
                "instances_tested": len(n_values),
                "conjecture_holds": False,
                "counterexample": f"Failed for n={n}, T(2,1)={T21}, T(1,2)={T12}"
            }

        results.append(T21)

    return {
        "metric_name": "T(2,1), T(1,2)",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = (sum((x - mean) ** 2 for x in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r is not None and r >= n_values[-1] ** (n_values[

    if all(r is not None and r >= n_values[-1] ** (n_values[-1] // 4) for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
```

```
    elif any(r is not None and r < n_values[-1] ** (n_values[-1] // 4) for r in results):
        first_failing_seed = seeds[results.index(next(r for r in results if r is not None and r <
        print(f"RESULT: FALSIFIED counterexample='T(2,1), T(1,2) < n^{n_values[-1]//4}' first_fail
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12ec44ec.py", line 99, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12ec44ec.py", line 72, in run_trial
    T21, T12 = get_tutte_values(graph, n // 2 + 1)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12ec44ec.py", line 61, in get_tutte_val
    T21 = tutte_polynomial(graph)
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12ec44ec.py", line 29, in tutte_polynom
    if (graph,) in memo:
       ^^^^^^^^^^^^^^^^
TypeError: unhashable type: 'collections.defaultdict'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to unhashable defaultdict graph input | next: Fix graph representation to use hashable structures like frozenset for memoization

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111375
2	propose	ollama_remote	qwen3:8b	0	0	97587
3	propose	ollama_remote	qwen3:8b	0	0	100543
4	preregistration	ollama_remote	qwen3:8b	0	0	24091
5	novelty	ollama_remote	qwen3:8b	0	0	20691
6	novelty	ollama_remote	qwen3:8b	0	0	15512

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18332
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11169
9	judge	ollama_remote	qwen3:8b	0	0	16308

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 415608 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/954a5c8e8ffd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/954a5c8e8ffd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/954a5c8e8ffd.tar.gz` (if generated)